arXiv:2212.05178v1 [cs.LG] 10 Dec 2022

State-Regularized Recurrent Neural Networks to Extract Automata and Explain Predictions

Cheng Wang†, *Member IEEE,* Carolin Lawrence, Mathias Niepert

Abstract—Recurrent neural networks are a widely used class of neural architectures. They have, however, two shortcomings. First, they are often treated as black-box models and as such it is difficult to understand what exactly they learn as well as how they arrive at a particular prediction. Second, they tend to work poorly on sequences requiring long-term memorization, despite having this capacity in principle. We aim to address both shortcomings with a class of recurrent networks that use a stochastic state transition mechanism between cell applications. This mechanism, which we term state-regularization, makes RNNs transition between a finite set of learnable states. We evaluate state-regularized RNNs on (1) regular languages for the purpose of automata extraction; (2) non-regular languages such as balanced parentheses and palindromes where external memory is required; and (3) real-word sequence learning tasks for sentiment analysis, visual object recognition and text categorisation. We show that state-regularization (a) simplifies the extraction of finite state automata that display an RNN's state transition dynamic; (b) forces RNNs to operate more like automata with external memory and less like finite state machines, which potentiality leads to a more structural memory; (c) leads to better interpretability and explainability of RNNs by leveraging the probabilistic finite state transition mechanism over time steps.

Index Terms—recurrent neural networks, memorization, automata extraction, state machine, interpretability, explainability.

1 Introduction

Recurrent neural networks (RNNs) have found their way into numerous applications. Still, RNNs have two shortcomings. First, it is difficult to understand what concretely RNNs learn. However, some applications require a close inspection of learned models before deployment and RNNs are more difficult to interpret than rule-based systems. There are a number of approaches for extracting deterministic finite automata (DFAs) from trained RNNs [22], [61], [62] as a means to analyze their behavior. These methods apply extraction algorithms after training and it remains challenging to determine whether the extracted DFA faithfully models the RNN's state transition behavior. Most extraction methods are rather complex, depend crucially on hyperparameter choices, and tend to be computationally costly. Second, RNNs tend to work poorly on input sequences requiring long-term memorization, despite having this ability in principle. Indeed, there is a growing body of work providing evidence, both empirically [3], [13], [56] and theoretically [1], [42], [74], that recurrent networks offer no benefit on longer sequences, at least under certain conditions. Intuitively, RNNs tend to operate more like DFAs with a large number of states, attempting to memorize all the information about the input sequence solely with their hidden states, and less like automata with external memory.

We propose state-regularized RNNs as a possible step towards addressing both of the aforementioned problems. State-regularized RNNs (SR-RNNs) are a class of recurrent networks that utilize a stochastic state transition mechanism between cell applications. The stochastic mechanism models a probabilistic state dynamic that lets the SR-RNNs transition between a finite number of learnable states. The parameters of the stochastic mechanism are trained jointly with the parameters of the base RNN.

SR-RNNs have several advantages over standard RNNs. First, instead of having to apply post-training DFA extraction, SR-RNNs determine their (probabilistic and deterministic) state transition behavior more directly. We propose a method that extracts DFAs truly representing the state transition behavior of the underlying RNNs. Second, we hypothesize that the frequently-observed poor extrapolation behavior of RNNs is caused by memorization with hidden states. It is known that RNNs – even those with cell states or external memory – tend to memorize mainly with their hidden states and in an unstructured manner [28], [52]. We show that the state-regularization mechanism shifts representational power

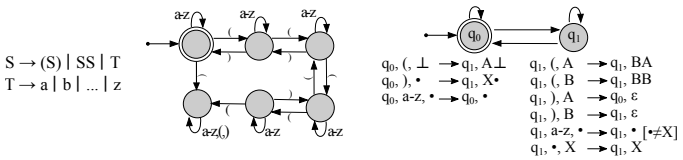


Fig. 1: (Left) The context-free grammar for the language of balanced parentheses (BP). (Center) A DFA that recognizes BP up to depth 4. (Right) A deterministic pushdown automaton (DPDA) that recognizes BP for all depths. The symbol • is a wildcard and stands for all possible tokens. The DPDA extrapolates to all sequences of BP, the DFA recognizes only those up to nesting depth 4.

- †*Work done while at NEC Laboratories Europe.*
- *Cheng Wang is with Amazon, Berlin, Germany. E-mail: dr.rer.nat.chengwang@gmail.com.*
- *Carolin Lawrence is with NEC Laboratories Europe, Heidelberg, Germany, E-mail: carolin.lawrence@neclab.eu.*
- *Mathias Niepert is with NEC Laboratories Europe, Heidelberg, Germany and the University of Stuttgart, Stuttgart, Germany, E-mail: mathias.niepert@neclab.eu.*
- *Correspondence author: Cheng Wang.*

Manuscript received November 1, 2020; revised November 11, 2022.

to memory components such as the cell state, resulting in improved extrapolation performance.

We support our hypotheses through experiments both on synthetic and real-world datasets. We explore the improvement of the extrapolation capabilities of SR-RNNs and closely investigate their memorization behavior. For state-regularized LSTMs, for instance, we observe that memorization can be shifted entirely from the hidden state to the cell state. For text and visual data, state-regularization provides more intuitive interpretations of the RNNs' behavior. A preliminary version of this work appeared in [59]

2 Background

SR-RNNs add state-regularization to RNNs and therefore we first define RNNs (Section 2.1). Furthermore, given an SR-RNN, we describe how a DFA can be extracted faithfully and thus we describe deterministic finite & pushdown automata (Section 2.2).

2.1 Recurrent Neural Networks (RNNs)

RNNs are powerful learning machines. Siegelmann and Sontag [48], [49], [50], for instance, proved that a variant of Elman-RNNs [15] can simulate a Turing machine. Given an input x_t at t-th time step:

$$\mathbf{h}_t = \sigma(\mathbf{W}_h x_t + \mathbf{U}_h \mathbf{h}_{t-1} + b_h), \quad (1)$$
$$\mathbf{y}_t = \sigma(\mathbf{W}_y x_t + b_y), \quad (2)$$

where $\mathbf{W}, \mathbf{U}, b_h, b_y$ are parameters, σ is the sigmoid activation function, $\mathbf{h}$ is the hidden representation and $\mathbf{y}$ the output. The key issue for an RNN is to learn to preserve information over long time steps due to vanishing gradients. To alleviate this, several RNN variations have been proposed. Two popular ones are Long Short-Term Memories (LSTMs) [29] and Gated Recurrent Units (GRUs) [8]. An LSTM consists of an input gate $\mathbf{i}$, a forget gate $\mathbf{f}$, a memory cell $\mathbf{c}$ and an output gate $\mathbf{o}$:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f x_t + \mathbf{U}_f \mathbf{h}_{t-1} + b_f), \quad (3)$$
$$\mathbf{i}_t = \sigma(\mathbf{W}_i x_t + \mathbf{U}_i \mathbf{h}_{t-1} + b_i), \quad (4)$$
$$\mathbf{o}_t = \sigma(\mathbf{W}_o x_t + \mathbf{U}_o \mathbf{h}_{t-1} + b_o), \quad (5)$$
$$\hat{\mathbf{c}}_t = \phi(\mathbf{W}_c x_t + \mathbf{U}_c \mathbf{h}_{t-1} + b_c), \quad (6)$$
$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \hat{\mathbf{c}}_t, \quad (7)$$
$$\mathbf{h}_t = \mathbf{o}_t \odot \phi(\mathbf{c}_t); \quad (8)$$

a GRU has an update gate $\mathbf{z}$ and an reset gate $\mathbf{r}$:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z x_t + \mathbf{U}_z \mathbf{h}_{t-1} + b_z), \quad (9)$$
$$\mathbf{r}_t = \sigma(\mathbf{W}_r x_t + \mathbf{U}_r \mathbf{h}_{t-1} + b_r), \quad (10)$$
$$\hat{\mathbf{h}}_t = \phi(\mathbf{W}_h x_t + \mathbf{r}_t \odot \mathbf{h}_{t-1} + b_h, \quad (11)$$
$$\mathbf{h}_t = \mathbf{z}_t \odot \hat{\mathbf{h}}_t + (1 - \mathbf{z}) \odot \mathbf{h}_{t-1}, \quad (12)$$

where ϕ is hyperbolic tangent function and $\odot$ is the element-wise multiplication operation. Recent work considers the more practical situation where RNNs have finite precision and linear computation time in their input length [63].

2.2 Deterministic Finite & Pushdown Automata

We provide some background on deterministic finite automata (DFAs) and deterministic pushdown automata (DPDAs) for two reasons. First, one contribution of our work is a method for extracting DFAs from RNNs. Second, the state regularization we propose is intended to make RNNs behave more like DPDAs and less like DFAs by limiting their ability to memorize with hidden states.

A DFA is a state machine that accepts or rejects sequences of tokens and produces one unique computation path for each input. Let Σ^* be the language over the alphabet Σ and let ϵ be the empty sequence. A DFA is a 5-tuple $(\mathcal{Q}, \Sigma, \delta, q_0, F)$ consisting of a finite set of states $\mathcal{Q}$, a finite set of input tokens Σ called the input alphabet, a transition functions $\delta : \mathcal{Q} \times \Sigma \to \mathcal{Q}$, a start state q_0 and a set of accept states $F \subseteq \mathcal{Q}$. A sequence w is accepted by the DFA if the application of the transition function, starting with q_0, leads to an accepting state. Figure 1 (center) depicts a DFA for the language of balanced parentheses (BP) up to depth 4. A language is regular if and only if it can be described by a DFA.

A pushdown automata (PDA) is defined as a 7-tuple $(\mathcal{Q}, \Sigma, \Gamma, \delta, q_0, \perp, F)$ consisting of a finite set of states $\mathcal{Q}$; a finite set of input tokens Σ called the input alphabet, a finite set of tokens Γ called the stack alphabet, a transition function $\delta \subseteq \mathcal{Q} \times (\Sigma \cup \epsilon) \times \Gamma \to \mathcal{Q} \times \Gamma^*$, a start state q_0, the initial stack symbol $\perp$, and a set of accepting states $F \subseteq \mathcal{Q}$. Computations of the PDA are applications of the transition relations. The computation starts in q_0 with the initial stack symbol $\perp$ on the stack and sequence w as input. The pushdown automaton accepts w if after reading w the automaton reaches an accepting state. Figure 1 (right) depicts a deterministic PDA for the language BP.

3 Related Work

Our SR-RNNs and applications relate to four different lines of work. First, we discuss existing works that examine how to extract DFAs from RNNs (Section 3.1). Second, we look at alternative options for regularizing RNNs (Section 3.2). Third, we describe other RNN extensions that modify the state or add an external memory (Section 3.3). Fourth, we list approaches to better understand RNNs and discuss how SR-RNNs can support with this effort (Section 3.4).

3.1 Extracting DFAs from RNNs

Extracting DFAs from RNNs goes back to work on first-generation RNNs in the 1990s [22], [71]. These methods perform a clustering of hidden states after the RNNs are trained [18], [22], [60]. Recent work introduced more sophisticated learning approaches to extract DFAs from LSTMs and GRUs [62]. The latter methods tend to be more successful in finding DFAs behaving similar to the underlying RNN. In contrast to all existing methods, SR-RNNs learn an explicit set of states which facilitates the extraction of DFAs from memory-less SR-RNNs by modelling exactly their state transition dynamics. A different line of work attempt to learn more interpretable RNNs [16], or rule-based classifiers from RNNs [44].

3.2 Regularizations of RNNs

There is a large body of work on regularization techniques for RNNs. Most of these adapt regularization approaches developed for feed-forward networks to the recurrent setting. Representative instances are dropout regularization [68], variational dropout [19], weight-dropped LSTMs [41], Zoneout [35] and noise injection [14]. Two approaches that can improve convergence and generalization capabilities are batch normalization [9] and weight initialization strategies [36] for RNNs. In contrast, the proposed SR-RNNs regularize the number of hidden states to a finite set of states. As a result, LSTMs with state regularization can learn in a more structural manner, which leads to improved generalization.

3.3 State and External Memory Extensions to RNNs

The work most similar to SR-RNNs are self-clustering RNNs [71]. These RNNs learn discretized states, that is, binary valued hidden state vectors, and it can be shown that these networks generalize better to longer input sequences. Contrary to self-clustering RNNs, we propose an end-to-end differentiable probabilistic state transition mechanism between cell applications.

Stochastic RNNs are a class of generative recurrent models for sequence data [4], [17], [23]. They model uncertainty in the hidden states of an RNN by introducing latent variables. In contrast to SR-RNNs, stochastic RNNs do not model probabilistic state transition dynamics. Hence, they do not address the problem of overfitting through hidden state memorization nor can they improve DFA extraction.

There are proposals for extending RNNs with various types of external memory. Representative examples are the neural Turing machine [24], improvements thereof [25], memory network [64], associative LSTM [12], and RNNs augmented with neural stacks, queues, and deques [26]. Contrary to these proposals, we do not augment RNNs with differentiable data structures but regularize RNNs to make better use of existing memory components such as the cell state. We hope, however, that differentiable neural computers could benefit from state-regularization.

3.4 Understanding RNNs

Approaches for understanding CNNs [51], [70], [72] have been explored extensively. Studies for interpreting and explaining RNNs are less common. [32] revealed the existence of interpretable LSTM cells with character-level language models. [38] visualized neural language models. [52] presented a visual analysis tool (namely, LSTMVIS) for visualizing the raw gate activations of LSTMs on understanding these hidden state dynamics over sequences. While the methods can identify the semantic correlations between hidden cells and abstract attributes or concepts, it is still not obvious how to explain the prediction for given inputs. One of the most recent methods from [44] describe a method to extract simple phrase patterns for determining LSTM predictions. With state-regularization we are able to increase the interpretability of RNNs by inspecting the probabilistic state transition over time steps and by directly extracting automata from trained RNN models.

4 State-Regularized Recurrent Networks

The standard recurrence of an RNN is $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$ where $\mathbf{h}_{t-1}$ is the hidden state vector at time $t-1$, and $\mathbf{h}_t$ and $\mathbf{x}_t$ are the hidden state and the input symbol at time t, respectively. We refer to RNNs whose unrolled cells are only connected through gated hidden states $\mathbf{h}$ as RNNs *without* ∞*-memory*. This is because values of gated hidden states $\mathbf{h}$ can only be in a particular interval, such as $[-1, 1]$ for `tanh` and not $(-\infty, \infty)$. This limits, in this case, the information flow between cells to values between -1 and 1 and, therefore, memorization has to be performed with fractional changes. The family of GRUs is without ∞-memory, while LSTMs have ∞-memory due to their cell state.

A cell of a state-regularized RNN (SR-RNN) consist of two components. The first component, which we refer to as the *recurrent component*, applies the function of a standard RNN cell

$$\mathbf{u}_t = f(\mathbf{h}_{t-1}, \mathbf{c}_{t-1}, \mathbf{x}_t). \tag{13}$$

For the sake of completeness, we include the cell state $\mathbf{c}$ here, which is absent in RNNs without ∞-memory.

We propose a second component which we refer to as *stochastic component*. The stochastic component is responsible for modeling the probabilistic state transitions that let the RNN transition implicitly between a finite number of states. Let d be the size of the hidden state vectors of the recurrent cells. Moreover, let $\Delta^D := \{\boldsymbol{\lambda} \in \mathbb{R}_+^D : \| \boldsymbol{\lambda} \| = 1\}$ be the $(D-1)$ probability simplex. The stochastic component maintains k learnable centroids $\mathbf{s}_1$, ..., $\mathbf{s}_k$ of size d which we often write as the column vectors of a matrix $\mathbf{S} \in \mathbb{R}^{d \times k}$. The weights of these centroids are global parameters shared among all cells. The stochastic component computes, at each time step t, a discrete probability distribution from the output $\mathbf{u}_t$ of the recurrent component and the centroids of the stochastic component

$$\boldsymbol{\alpha} = \omega(\mathbf{S}, \mathbf{u}_t) \text{ with } \boldsymbol{\alpha} \in \Delta^k. \tag{14}$$

Crucially, instances of ω should be differentiable to facilitate end-to-end training. Typical instances of the function ω are based on the dot-product, normalized into a probability distribution

$$\alpha_i = \frac{\exp((\mathbf{u}_t \cdot \mathbf{s}_i)/\tau)}{\sum_{i=1}^{k} \exp((\mathbf{u}_t \cdot \mathbf{s}_i)/\tau)} \tag{15}$$

Here, $\cdot$ is the inner product between two vectors and τ is a temperature parameter that can be used to anneal the probabilistic state transition behavior. The lower τ the more $\boldsymbol{\alpha}$ resembles the one-hot encoding of a centroid. The higher τ the more uniform $\boldsymbol{\alpha}$ becomes. Equation 15 is reminiscent of the equations of attentive mechanisms [2], [57]. However, instead of attending to the hidden states, SR-RNNs attend to the k centroids to compute transition probabilities. Each α_i is the probability of the RNN to transition to centroid (state) i given the vector $\mathbf{u}_t$ for which we write $p_{\mathbf{u}_t}(i) = \alpha_i$. The method has been recently introduced to estimate the uncertainty of RNN [58] and transformer models [45].

4.1 State Transition Mechanisms

The state transition dynamics of an SR-RNN is that of a probabilistic finite state machine. At each time step, when in state $\mathbf{h}_{t-1}$ and reading input symbol $\mathbf{x}_t$, the probability

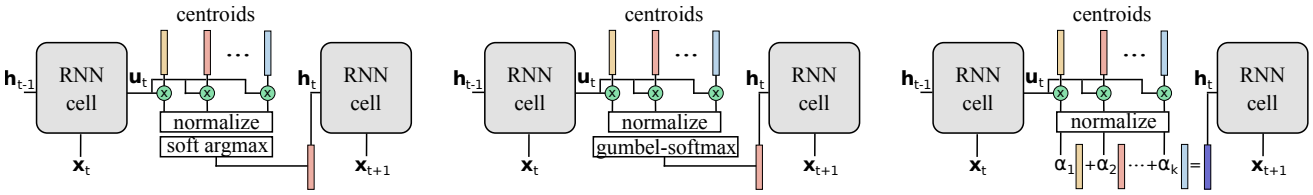


Fig. 2: Three possible instances of an SR-RNN corresponding to equations 17, 19 and 20 respectively.

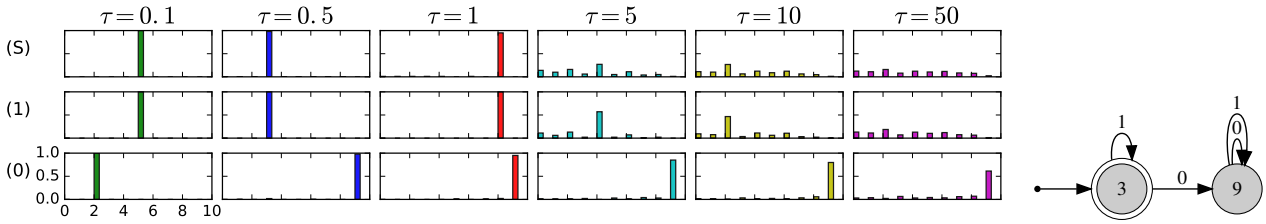


Fig. 3: (Left) State transition probabilities for the SR-GRU learned from the data for the Tomita 1 grammar, for temperatures τ and input sequence $[10]$. S is the start token. Centroids are listed on x-axis, probabilities on y-axis. Up to temperature $\tau = 1$ the behavior of the trained SR-GRUs is almost identical to that of a DFA. Despite the availability of $k = 10$ centroids, the trained SR-GRUs use the minimal set of states for $\tau \leq 1$. (Right) The extracted DFA for Tomita grammar 1 and temperature $\tau = 0.5$.

for transitioning to state $\mathbf{s}_i$ is α_i. Hence, in its second phase, the stochastic component computes the hidden state $\mathbf{h}_t$ at time step t from the distribution $\boldsymbol{\alpha}$ and the matrix $\mathbf{S}$ with a (possibly stochastic) mapping $h : \Delta^k \times \mathbb{R}^{d \times k} \rightarrow \mathbb{R}^d$. Thus, $\mathbf{h}_t = h(\boldsymbol{\alpha}, \mathbf{S})$. An instance of h is to

$$\text{sample } j \sim p_{\mathbf{u}_t} \text{ and set } \mathbf{h}_t = \mathbf{s}_j. \tag{16}$$

Assigning a particular centroid to be a hidden state, i.e. $\mathbf{h}_t = \mathbf{s}_j$, is equivalent to an one-hot encoding of centroids. However, the direct application of $\arg\max$ renders the SR-RNN not end-to-end differentiable. To ensure end-to-end differentiability, there are three possible alternative to represent states $\mathbf{s}_j$: (1) *soft argmax*, (2) *gumbel-softmax* [27], [30], [33] and (3) *mixture of centroids*.

First, a soft and differentiable version of $\arg\max$ can be achieved using Equation 15 with a low temperature parameter τ and as τ approaches 0, $\boldsymbol{\alpha}$ approximates a one-hot distribution and is differentiable.

$$\boldsymbol{\alpha} = \text{ONE_HOT}\big(\arg\max_i\big(\frac{\exp\left((\mathbf{u}_t \cdot \mathbf{s}_i)/\tau\right)}{\sum_{j=1}^{k} \exp\left((\mathbf{u}_t \cdot \mathbf{s}_j)/\tau\right)}\big)\big), \tag{17}$$

when τ approaches 0, the $\boldsymbol{\alpha}$ approximates one-hot distribution, but it offers differentiablity to SR-RNN.

Second, with the gumbel-trick it is possible to draw samples $\boldsymbol{z}$ from a categorical distribution given by paramaters $\boldsymbol{\theta}$, that is,

$$\boldsymbol{z} = \text{ONE_HOT}\big(\arg\max_i[g_i + (\mathbf{u}_t \cdot \mathbf{s}_i)]\big), i \in [1 \dots k], \tag{18}$$

where g_i are i.i.d. samples from the $\text{GUMBEL}(0, 1)$, that is, $g = -\log(-\log(u)), u \sim \text{UNIFORM}(0, 1)$. Because the $\arg\max$ operator breaks end-to-end differentiability, the categorical distribution $\boldsymbol{z}$ can be approximated using the differentiable softmax function [30], [33]. This enables us to draw a k-dimensional sample vector $\boldsymbol{\alpha} \in \Delta^{k-1}$, where Δ^{k-1} is the $(k-1)$-dimensional probability simplex. Each instance $\alpha_i \in \boldsymbol{\alpha}$ is assigned a probability, that is,

$$\alpha_i = \frac{\exp(((\mathbf{u}_t \cdot \mathbf{s}_i) + g_i)/\tau)}{\sum_{j=1}^{k} \exp((\mathbf{u}_t \cdot \mathbf{s}_j) + g_j)/\tau)}, i \in [1 \dots k], \tag{19}$$

where τ is a temperature and $\boldsymbol{\alpha}$ approaches $\boldsymbol{z}$ as $\tau \rightarrow 0$. Recently [58] showed this approach is able to learn better-calibrated models.

Third, it is possible to set the hidden state $\mathbf{h}_t$ to be the probabilistic *mixture of the centroids*

$$\mathbf{h}_t = \sum_{i=1}^{k} \alpha_i \mathbf{s}_i. \tag{20}$$

Every internal state $\mathbf{h}$ of the SR-RNN, therefore, is computed as a weighted sum $\mathbf{h} = \alpha_1 \mathbf{s}_1 + ... + \alpha_k \mathbf{s}_k$ of the centroids $\mathbf{s}_1, ..., \mathbf{s}_k$ with $\boldsymbol{\alpha} \in \Delta^k$. Here, h is a smoothed variant in contrast to a hard assignment to one of the centroids. Figure 2 depicts the three variants of the proposed SR-RNNs.

The probabilistic state transition mechanism is also applicable when RNNs have more than one hidden layer. In RNNs with $l > 1$ hidden layers, every such layer can maintain its own centroids and stochastic component. In this case, a global state of the SR-RNN is an l-tuple, with the lth argument of the tuple corresponding to the centroids of the lth layer.

Even though we have augmented the original RNN with additional learnable parameter vectors, we are actually constraining the SR-RNN to output hidden state vectors that are similar to the centroids. For lower temperatures and smaller values for k, the ability of the SR-RNN to memorize with its hidden states is increasingly impoverished. We argue that this behavior is beneficial for three reasons:

- First, it makes the extraction of interpretable DFAs from memory-less SR-RNNs straight-forward. Instead of applying post-training DFA extraction as in previous work [61], [62], we extract the true underlying DFA directly from the SR-RNN. Specifically, SR-RNNs don't

need an intermediate step that extracts representations from pre-trained RNN models and performs clustering e.g., k-means over the representations. This is automatically done by equations (14),(15),(16) and Algorithm1[1]
- Second, we hypothesize that overfitting in the context of RNNs is often caused by memorization via hidden states. Indeed, we show that regularizing the state space pushes representational power to memory components such as the cell state of an LSTM, resulting in improved extrapolation behavior.
- Third, the values of hidden states tend to increase in magnitude with the length of the input sequence, a behavior that has been termed *drifting* [71]. The proposed state regularization stabilizes the hidden states for longer sequences.

Next, let us explore some of the theoretical properties of the proposed mechanism. We show that the addition of the stochastic component, when capturing the complete information flow between cells as, for instance, in the case of GRUs, makes the resulting RNN's state transition behavior identical to that of a probabilistic finite state machine.

Theorem 4.1. *The state transition behavior of an* SR-RNN *without ∞-memory using Equation 16 is identical to that of a probabilistic finite automaton.*

Theorem 4.2. *For $\tau \to 0$ the state transition behavior of an* SR-RNN *without ∞-memory (using Equations 16 or 20) is equivalent to that of a deterministic finite automaton.*

The proofs of the theorems are part of the appendix.

4.2 Learning DFAs with State-Regularized RNNs

Extracting DFAs from RNNs is motivated by applications where a thorough understanding of learned neural models is required before deployment. SR-RNNs maintain a set of learnable states and compute and explicitly follow state transition probabilities. It is possible, therefore, to extract finite-state transition functions that truly model the underlying state dynamics of the SR-RNN. The centroids do not have to be extracted from a clustering of a number of observed hidden states but can be read off of the trained model. This renders the extraction also more efficient.

We developed two possible approaches, *transition counts*-based and *mean transition probability*-based, to extract DFAs. First, we adapt previous work [46], [61] to construct the transition function of an SR-RNN. We begin with the start token of an input sequence, compute the transition probabilities $\boldsymbol{\alpha}$, and move the SR-RNN to the highest probability state. We continue this process until we have seen the last input token. By doing this, we get a count of transitions from every state $\mathbf{s}_i$ and input token $a \in \Sigma$ to the following states (including self-loops). After obtaining the transition counts, we keep only the most frequent transitions and discard all other transitions. Concretely, Algorithm 1 presents the pseudo-code of DFA extraction, where $\Phi[(\mathbf{s}_i, \mathbf{x}_t, \mathbf{s}_j)]$ is a dictionary of transitions and counts, where the tuple $(\mathbf{s}_i, \mathbf{x}_t, \mathbf{s}_j)$ denotes a transition from centroid $\mathbf{s}_i$ to $\mathbf{s}_j$ given an input symbol $\mathbf{x}_t$ and δ is the returned transition function.

1. It can be integrated to the training loop and output extract DFA at every epoch. Importantly, we don't need to explicity tune the number of clusters k.

Algorithm 1 Learning transition function (counts-based)

Input: model $\mathbf{M}$, dataset $\mathbf{D}$, alphabet Σ, start token s
Output: transition function δ

* *the transition prob. of start token* *\
$\Phi[(\mathbf{s}_i, \mathbf{x}_t, \mathbf{s}_j)] = 0, \;\; i, j \in \{1, ..., k\}, \;\; \mathbf{x}_t \in \Sigma$
$\boldsymbol{\alpha} = \mathbf{M}(s), \boldsymbol{\alpha} = \{\alpha_i\}, i \in \{1, ..., k\}$
* *select the next state* *\
$i = \arg\max_{i \in \{1,...,k\}}[\boldsymbol{\alpha}], \;\; \mathbf{s}_{start} = \mathbf{s}_i$
* *compute and update for each input symbol* *\
for $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_T) \in \mathbf{D}$ **do**
  for $t \in [1, ..., T]$ **do**
    $j = \arg\max_{i \in \{1,...,k\}}[\mathbf{M}(\mathbf{x}_t)], \;\; \mathbf{s}_{end} = \mathbf{s}_j$
    $\Phi[(\mathbf{s}_{start}, \mathbf{x}_t, \mathbf{s}_{end})] \leftarrow \Phi[(\mathbf{s}_{start}, \mathbf{x}_t, \mathbf{s}_{end})] + 1$
    $\mathbf{s}_{start} = \mathbf{s}_{end}$
  end for
end for
* *compute transition function based on transition counts* *\
for $i, j \in \{1, ..., k\}$ and $\mathbf{x}_t \in \Sigma$ **do**
  $\delta(\mathbf{s}_i, \mathbf{x}_t) = \arg\max_{j \in \{1,...,k\}} \Phi[(\mathbf{s}_i, \mathbf{x}_t, \mathbf{s}_j)]$
end for

Alternatively, we can learn the transition function based on mean transition probability, rather than transition counts. This can be achieved by computing the mean transition probability:

$$\Phi[(\mathbf{s}_{start}, \mathbf{x}_t, \mathbf{s}_{end})] \leftarrow \Phi[(\mathbf{s}_{start}, \mathbf{x}_t, \mathbf{s}_{end})] + \max[\boldsymbol{\alpha}]. \quad (21)$$

In this case, δ is learned by computing the maximum mean transition probability

$$\delta(\mathbf{s}_i, \mathbf{x}_t) = \underset{j \in \{1,...,k\}}{\arg\max} \, \textsc{Mean}\left(\Phi[(\mathbf{s}_i, \mathbf{x}_t, \mathbf{s}_j)]\right). \quad (22)$$

As a corollary of Theorem 4.2, we have that, for $\tau \to 0$, the extracted transition function becomes increasingly identical to the transition function of the DFA learned by the SR-RNN. Figure 3 shows that for a wide range of temperatures (including the standard softmax temperature $\tau = 1$) the transition behavior of an SR-GRU is identical to that of a DFA, a behavior we can show to be common when SR-RNNs are trained on regular languages.

4.3 Learning Non-Regular Languages with State-Regularized LSTMs

For more complex languages, such as context-free languages, RNNs that behave like DFAs generalize poorly to longer sequences. The DPDA shown in Figure 1, for instance, correctly recognizes the language of BP, while the DFA only recognizes it up to nesting depth 4. We want to encourage RNNs with memory to behave more like DPDAs and less like DFAs. The transition function δ of a DPDA takes (a) the current state, (b) the current top stack symbol, and (c) the current input symbol and maps these inputs to (1) a new state and (2) a replacement of the top stack symbol (see Section 2). Hence, to allow an SR-RNN, such as the SR-LSTM, to operate in a manner similar to a DPDA we need to give the RNNs access to these three inputs when deciding what to forget from and what to add to the memory. Precisely this is accomplished for LSTMs with peephole connections [21].

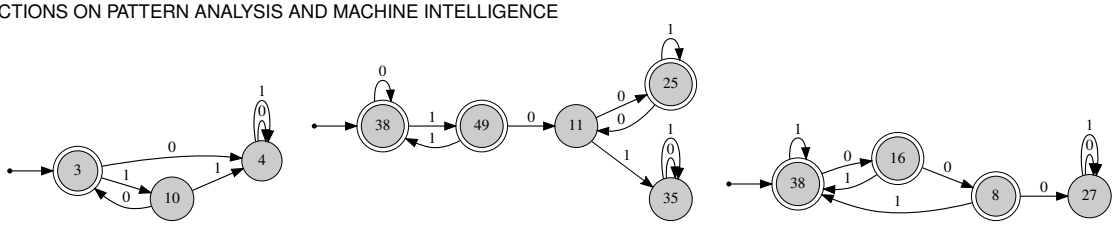


Fig. 4: Extracted DFAs of the Tomita grammars 2-4. All DFAs are correct. The state numbers correspond to the index of the learned SR-GRU centroids.

Dataset		Large Dataset			Small Dataset	
Models	LSTM	SR-LSTM	SR-LSTM-P	LSTM	SR-LSTM	SR-LSTM-P
$d \in [1, 10]$, $l \leq 100$	0.005	0.038	**0.000**	0.068	0.037	**0.017**
$d \in [10, 20]$, $l \leq 100$	0.334	0.255	**0.001**	0.472	0.347	**0.189**
$d \in [10, 20]$, $l \leq 200$	0.341	0.313	**0.003**	0.479	0.352	**0.196**
$d = 5$, $l \leq 200$	0.002	0.044	**0.000**	0.042	0.028	**0.015**
$d = 10$, $l \leq 200$	0.207	0.227	**0.004**	0.409	0.279	**0.138**
$d = 20$, $l \leq 1000$	0.543	0.540	**0.020**	0.519	0.508	**0.380**

TABLE 1: Error rates for the balanced parentheses (BP) test sets (d=depth, l=length, k=5 centroids, the training depth ≤ 5).

Concretely, to update the memory, the cell state of a LSTM incorporates forget, input and output gates:

$$\mathbf{f}_t = \sigma\big(\mathbf{W}^f\mathbf{x}_t + \mathbf{R}^f\mathbf{h}_{t-1} + \mathbf{p}^f \odot \mathbf{c}_{t-1} + \mathbf{b}^f\big), \quad (23)$$

$$\mathbf{i}_t = \sigma\big(\mathbf{W}^i\mathbf{x}_t + \mathbf{R}^i\mathbf{h}_{t-1} + \mathbf{p}^i \odot \mathbf{c}_{t-1} + \mathbf{b}^i\big), \quad (24)$$

$$\mathbf{o}_t = \sigma\big(\mathbf{W}^o\mathbf{x}_t + \mathbf{R}^o\mathbf{h}_{t-1} + \mathbf{p}^o \odot \mathbf{c}_t + \mathbf{b}^o\big), \quad (25)$$

where $\mathbf{h}_{t-1}$ is the output of the previous cell's stochastic component; $\mathbf{W}$s and $\mathbf{R}$s are the matrices of the original LSTM; the $\mathbf{p}$s are the parameters of the peephole connections; and $\odot$ is the elementwise multiplication. We show empirically that the resulting SR-LSTM-P operates like a DPDA, incorporating the current cell state when making decisions about changes to the next cell state.

4.4 Practical Considerations

Implementing SR-RNNs only requires extending existing RNN cells with a stochastic component. We have found the use of start and end tokens to be beneficial. The start token is used to transition the SR-RNN to a centroid representing the start state which then does not have to be fixed a priori. The end token is used to perform one more cell application but without applying the stochastic component before a classification layer. The end token lets the SR-RNN consider both the cell state and the hidden state to make the accept/reject decision. We find that a temperature of $\tau = 1$ (standard softmax) and an initialization of the centroids with values sampled uniformly from $[-0.5, 0.5]$ work well across different datasets.

4.5 Understanding RNN Models and Predictions

State-regularization provides new ways to interpret the working of RNNs. Since SR-RNNs have a finite set of states, we can use the observed transition probabilities to visualize their behavior. We argue that the proposed probabilistic state transition mechanism helps to understand RNN in: (1) model interpretation, what the RNN models learned from training data and (2) prediction explanation, the explanation for a specific prediction. We use the concept and definition of interpretation and explaination from Montavona et al. [43].

4.5.1 Extracting RNN Model Prototypes

To understand what RNN models learn on training data, we are interested in what each learned centroid can represent, because the learning of a centroid is essentially a "prototype-based clustering" which is similar to learning vector quantization (LVQ) [34]. When RNN models are trained in a supervised manner, the centroids can be learned to represent the semantic meaning of each categorical class. To represent centroids with most representative inputs, we keep track of the input with highest probability for each centroid. The set of words or pixels with highest transition probability are used to represent a specific centroid. We summarize the procedure in Algorithm 2.

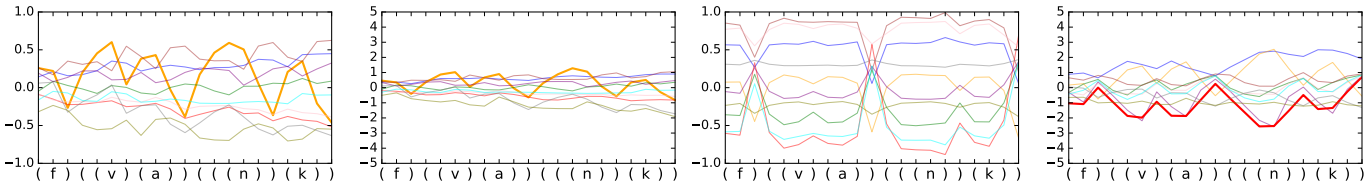
Algorithm 2 Generating model prototype

Input: trained model $\mathbf{M}$, training dataset $\mathbf{D}_{train}$
Output: the top N prototypical words for each centroid $V_i = \{x\}_i^N, i \in \{1, ..., k\}$

```
\* initialise an empty set for each centroid *\
s_i = {}, i ∈ {1, ..., k}
for x = (x_1, x_2, ..., x_T) ∈ D_train do
    for t ∈ [1, ..., T] do
        \* select the centroid that word x_t has highest prob. *\
        j = arg max_{i∈{1,...,k}} [M(x_t)]
        \* update the top N words for j-th centroid *\
        V_j.UPDATE(x_t)
    end for
end for
```

4.5.2 Explaining RNN Predictions

When applying a trained model to predict on test samples, it is often difficult to understand how the model arrived at the prediction. Probabilistic state transition offers a way to highlight the input symbols which trigger a high transition probability to each centroid. This allows us to highlight the inputs which are highly relevant for the final predictions. We summarize this in Algorithm 3.



(a) $\mathbf{h}_t$ of the LSTM (b) $\mathbf{c}_t$ of the LSTM (c) $\mathbf{h}_t$ of the SR-LSTM-P (d) $\mathbf{c}_t$ of the SR-LSTM-P

Fig. 5: Visualization of hidden state $\mathbf{h}_t$ and cell state $\mathbf{c}_t$ of the LSTM and the SR-LSTM-P for a specific input sequence from BP. Each color corresponds to one of 10 hidden units. The LSTM memorizes the number of open parentheses both in the hidden and, to a lesser extent, in the cell state (bold yellow lines). The memorization is not accomplished with saturated gate outputs and a drift is observable for both vectors. The SR-LSTM-P maintains two distinct hidden states (accept and reject) and does not visibly memorize counts through its hidden states. The cell state is used to cleanly memorize the number of open parentheses (bold red line) with saturated gate outputs (± 1). For SR-LSTM-P, a state vector drift is not observable; solutions with less drift generalize better [20].

Algorithm 3 Explaining model prediction

Input: model $\mathbf{M}$, a test sample $U = \{u_t\}_1^T$, vocabulary Σ (in language task)
Output: model prediction y and its explanation.

1: * *initialise an empty word-centroid transition matrix* P_t^i, $i \in \{1, ..., k\}$ *\
2: * *compute transition prob. for each word* u *\
3: **for** $u = (u_1, u_2, ..., u_T) \in U$ **do**
4: * *output* y, *last centroid index* j, *and* P_t^i *\
5: $y, j, P_t^i \Leftarrow \mathbf{M}(u)$
6: *generate explanations (e.g.,heatmap or highlight words) with* $\{u_t\}_1^T, \{P_t^j\}_1^T, \Sigma$ by mapping P_j^t to u_t.
7: **end for**

Number of centroids	$k=2$	$k=5$	$k=10$	$k=50$
$d \in [1,10], l \leq 100$	0.019	**0.017**	0.021	0.034
$d \in [10,20], l \leq 100$	**0.096**	0.189	0.205	0.192
$d \in [10,20], l \leq 200$	**0.097**	0.196	0.213	0.191
$d = 5, l \leq 200$	0.014	0.015	**0.012**	0.047
$d = 10, l \leq 200$	**0.038**	0.138	0.154	0.128
$d = 20, l \leq 1000$	0.399	**0.380**	0.432	0.410

TABLE 2: Error rates of the SR-LSTM-P on the small BP test data for various numbers of centroids k (d=depth, l=length).

5 Experiments

We conduct four types of experiments to investigate our hypotheses. First, we apply a simple algorithm for extracting DFAs and assess to what extent the true DFAs can be recovered from input data. Second, we compare the behavior of LSTMs and state-regularized LSTM on non-regular languages, such as the languages of balanced parentheses and palindromes. Third, we investigate the performance of state-regularized LSTMs on non-synthetic datasets. Last, we visualize the probabilistic state transitions to understand RNN models and explain their predictions.

Unless otherwise indicated we always (a) use single-layer RNNs, (b) learn an embedding for input tokens before feeding it to the RNNs, (c) apply ADADELTA [69] for regular language and RMSPROP [54] with a learning rate of 0.01 and momentum of 0.9 for the rest; (d) do not use dropout or batch normalization of any kind; and (e) use state-regularized RNNs based on Equations 15 and 20 with a temperature of $\tau = 1$ (standard softmax). We implemented SR-RNNs with Theano [53] [2]. All experiments were performed on a single Titan Xp with 12G memory. The hyper-parameter were tuned to make sure the vanilla RNNs achieve the best performance.

5.1 Regular Languages and DFA Extraction

We evaluate the DFA extraction algorithm for SR-RNNs on RNNs trained on the Tomita grammars [55], which have been used as benchmarks in previous work [61], [62]. We use available code [62] to generate training and test data for the regular languages. We first trained a single-layer GRU with 100 units on the data. We use GRUs since they are memoryless and, hence, Theorem 4.2 applies. Whenever the GRU converged within 1 hour to a training accuracy of 100%, we also trained an SR-GRU based on Equations 15 and 20 with $k = 50$ and $\tau = 1$. This was the case for the Grammars 1-4 and 7. The difference in time to convergence between the vanilla GRU and the SR-GRU was negligible. We applied the transition function extraction outlined in Section 4.2. In all cases, we could recover the minimal and correct DFA corresponding to the grammars. Figure 4 depicts the DFAs for Grammars 2-4 extracted by our approach. Remarkably, even though we provide more centroids (possible states; here $k = 50$) the SR-GRU only utilizes the required minimal number of states for each of the grammars. Figure 3 visualizes the transition probabilities for different temperatures and $k = 10$ for Grammar 1. The numbers on the states correspond directly to the centroid numbers of the learned SR-GRU. One can observe that the probabilities are spiky, causing the SR-GRU to behave like a DFA for $\tau \leq 1$.

5.2 Non-regular Languages

We conducted experiments on non-regular languages where external memorization is required. This allows us to investigate whether SR-LSTM behave more like DPDAs and, therefore, extrapolate to longer sequences. To this end, we used the context-free language "balanced parentheses" (BP; see Figure 1 (left)) over the alphabet $\Sigma = \{a, ..., z, (,)\}$, used in previous work [62]. We created two datasets for BP. A large one with 22,286 training sequences (positive: 13,025; negative: 9,261) and 6,704 validation sequences (positive: 3,582; negative: 3,122). The small dataset consists of 1,008 training sequences (positive: 601; negative: 407), and 268

2. http://www.deeplearning.net/software/theano/

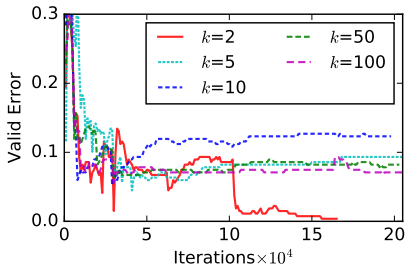


Fig. 6: SR-LSTM-P curves on the small BP validation data.

validation sequences (positive: 142; negative: 126). Both datasets have 1000 test samples. The training sequences have nesting depths $d \in [1, 5]$, the validation sequences $d \in [6, 10]$ and the test sequences $d \in [0, 20]$. We trained the LSTM and the SR-RNNs using curriculum learning as in previous work [62], [67] and using the validation error as stopping criterion. We then applied the trained models to unseen sequences. Table 1 lists the results on 1,000 test sequences with the respective depths and lengths. The results show that both SR-LSTM and SR-LSTM-Ps extrapolate better on longer sequences and sequences with deeper nesting. Moreover, the SR-LSTM-P performs almost perfectly on the large data, indicating that peephole connections are indeed beneficial.

To explore the effect of the hyperparameter k, that is, the number of centroids of the SR-RNNs, we ran experiments on the small BP dataset varying k and keeping everything else the same. Table 2 lists the error rates and Figure 6 the error curves on the validation data for the SR-LSTM-P and different values of k. While two centroids ($k = 2$) result in the best error rates for most sequence types, the differences are not very pronounced. This indicates that the SR-LSTM-P is robust to changes in the hyperparameter k. A close inspection of the transition probabilities reveals that the SR-LSTM-P mostly utilizes two states, independent of the value of k. These two states are used as accept and reject states. These results show that SR-RNNs generalize and tend to utilize a minimal set states similar to DPDAs.

A major hypothesis of ours is that the state-regularization encourages RNNs to operate more like DPDAs. To explore this hypothesis, we trained an SR-LSTM-P with 10 units on the BP data and visualized both the hidden state $\mathbf{h}_t$ and the cell state $\mathbf{c}_t$ for various input sequences. Similar state visualizations have been used in previous work [52], [63]. Figure 5 plots the hidden and cell states for a specific input, where each color corresponds to a dimension in the respective state vectors. As hypothesized, the LSTM relies primarily on its hidden states for memorization. The SR-LSTM-P, on the other hand, does not use its hidden states for memorization. Instead it utilizes two main states (accept and reject) and memorizes the nesting depth cleanly in the cell state. The visualization also shows a drifting behavior for the LSTM, in line with observations made for first-generation RNNs [71]. Drifting is not observable for the SR-LSTM-P.

We also performed experiments for the non-regular language ww^{-1} (Palindromes) [47] over the alphabet $\Sigma = \{a, ..., z\}$. We follow the same experiment setup as for BP.Results are presented in Table 3. This experiment adds evidence for the improved generalization and memorization behavior of state-regularized LSTMs over vanilla LSTMs (with peepholes).

Max Length	100	200	500
LSTM	31.2	42.0	47.7
LSTM-P	28.4	36.2	41.5
SR-LSTM	28.0	36.0	44.6
SR-LSTM-P	**10.5**	**16.7**	**29.8**

TABLE 3: Error rates in % on sequences of varying lengths from the Palindrome test set.

5.3 Performance on Real-world Tasks

Next we test our state regularization on real world datasets, namely on the tasks for sentiment analysis (Section 5.3.1) and digit recognition (Section 5.3.2).

5.3.1 Sentiment Analysis

We evaluated state-regularized LSTMs on the IMDB review dataset [39]. It consists of 100k movie reviews (25k training, 25k test, and 50k unlabeled). We used only the labeled training and test reviews. Each review is labeled as *positive* or *negative*. Table 4 lists the results. The SR-LSTM-P is competitive with state of the art methods that also do not use the unlabeled data.

Methods	Error
use additional unlabeled data	
Full+unlabelled+BoW ([39])	11.1
LSTM with tuning and dropout ([11])	13.50
LM-LSTM+unlabelled ([11])	7.6
SA-LSTM+unlabelled ([11])	7.2
do not use additional unlabeled data	
seq2-bown-CNN ([31])	14.7
Vartional Dropout ([19])[3]	10.56
WRRBM+BoW(bnc) ([10])	10.8
JumpLSTM ([66])	10.6
LSTM	10.1
LSTM-P	10.3
SR-LSTM ($k = 10$)	9.4
SR-LSTM-P ($k = 10$, eq. 17)	11.1
SR-LSTM-P ($k = 10$, eq. 19)	11.0
SR-LSTM-P ($k = 10$, eq. 20)	**9.2**
SR-LSTM-P ($k = 50$)	9.8

TABLE 4: Test error rates (%) on IMDB.

5.3.2 Pixel-by-Pixel MNIST

We also explored the impact of state-regularization on pixel-by-pixel MNIST [36], [37]. Here, the 784 pixels of MNIST images are fed to RNNs one by one for classification. This requires the ability to memorize long-term dependencies. Table 5 shows the results. The classification function has the final hidden and cell state as input. Our SR-LSTM-Ps do not use dropout, batch normalization, sophisticated weight-initialization, and are based on a simple single-layer LSTM. We can observe that SR-LSTM-Ps achieve competitive results, outperforming the vanilla LSTM and LSTM-P. We also conducted additional experiments with state-regularization on vanilla RNNs and GRUs (with the same number of hidden units as SR-LSTM-Ps). We achieve 31.9 and 13.6 test error,

Methods	Error
IRNN [36]	3.0
URNN [1]	4.9
Full URNN [65]	2.5
sTANH-RNN [73]	1.9
Skip LSTM [6]	2.7
r-LSTM Full BP [56]	1.6
BN-LSTM [9]	1.0
Dilated GRU [7]	**0.8**
LSTM	2.3
LSTM-P	1.5
SR-LSTM ($k = 100$)	1.4
SR-LSTM-P ($k = 100$, eq.17)	4.1
SR-LSTM-P ($k = 100$, eq.19)	6.7
SR-LSTM-P ($k = 100$, eq. 20)	**0.8**
SR-LSTM-P ($k = 50$)	1.4

TABLE 5: Test error (%) on the sequential MNIST.

cent.	words with top-4 highest transition probabilities
1	but (0.97) hadn (0.91) college (0.87) even (0.85)
2	not (1.0) or (1.0) italian (1.0) never (0.99)
3	loved (1.0) definitely (1.0) 8 (0.99) realistic (0.99)
4	no (1.0) worst (1.0) terrible (1.0) poorly (1.0)

TABLE 6: The learned centroids and their prototypical words with the top-4 highest transition probabilities on the IMDB dataset. This interprets the SR-LSTM-P model with centroids. The 3rd (4th) centroid is "positive" ("negative").

respectively, for SR-RNNs and SR-GRUs, which is worse than the results for the SR-LSTM-Ps. On MINST both networks failed to converge with same number of training epochs. This suggests the importance of a cell state and ∞-memory which we regularized to be used in a more structured manner.

6 Understanding RNNs with Probabilistic State Transitions

In this section, we visualize the learned centroids and the corresponding transition probabilities to understand the working of RNNs. We use the models trained on IMDB and sequential MNIST from the previous section. Additionally, we train a new SR-RNN model on 20NewsGroup dataset.[4] The dataset consists of 18,846 samples, among which 15,076 samples are used for training, and 3,770 samples are used for testing. The task is to classify text document into 20 categories. We built a vocabulary with a size of 10,003 and each word is encoded as one-hot representation. SR-LSTM is able to achieve classification accuracy of 85.6%. We also trained an SR-RNN ($k = 20$) on a more clean version of the dataset by removing header and footnotes, which achieves an accuracy of 65.7%.

6.1 Understanding RNN Models

Table 6 lists, for each state (centroid), the word with the top transition probabilities leading to this state. Here the SR-RNN is trained with $k = 5$ centroids. As we can see, only a limited number of centroids are meaningful, for example, the 3rd and 4th represent positive and negative sentiment[5].

Figure 7 presents the prototypes for digits 0, 3, 7. Interestingly, we also find the transition functions for each learned centroid are more like the "kernels" in CNNs. Each centroid intends to capture the different types of features. For example, the 7th centroid pays most of its attention to the left part of digits, and the 2nd centroid intends to capture the bold property of digits.

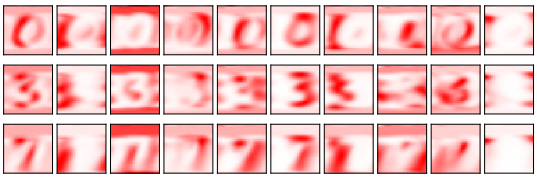


Fig. 7: Visualization of the mean transition probability to centroid of SR-RNNs ($k = 10$ centroids) models on the Sequential MNIST. The mean transition probabilities can be seen as a "categorical prototype" for each digit class. Each row represents a digit class and each column depicts the "categorical prototype" .

Table 7 shows the prototypical words for some categories in the 20NewsGroup dataset and the corresponding transition probabilities. As shown, the selected words with the highest probabilities are quite sensible (representative) to represent the corresponding class. For example, "domain, information, source, core, message" for representing class label "comp.windows.x". To some extent, this is similar to the topical words generated by topic models (e.g., Latent Dirichlet Allocation (LDA) [5]). Differently, SR-RNNs learn the typical words from sequential text data.

6.2 Explaining RNN Predictions

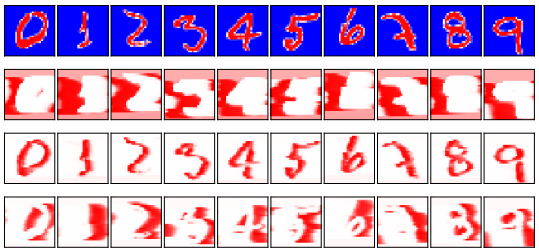


Fig. 8: For given test samples (top), SR-RNN ($k = 10$) gives the correct predictions on the sequential MNIST. To find the evidences to explain those predictions, we can look at the highest transition probabilities which lead to correct predictions. The 2nd to 4th rows present the highest transition probabilities to the 1st (top), the 3rd (middle) and the 4th centroid respectively.

Figure 8 gives the visual explanations for RNNs predictions on 10 randomly selected MNIST test samples. Table 8 presents the explanations of two sample texts from

3. The number is taken from [58]
4. http://qwone.com/~jason/20Newsgroups/

5. As each centroid $\mathbf{s}$ has same dimension as hidden state $\mathbf{h}$, we use learned classifier (sofmax layer) to classify centroid so as to decide its categorical label, e.g. positive or negative. If an input word has high transition (>0.5) probability to positive centroid, the word is treated as positive as well.

concept	the top prototypical words and transition probabilities				
talk.politics.misc	comment (0.58)	damage (0.53)	right (0.51)	drug (0.50)	obligation (0.49)
talk.politics.mideast	state (0.55)	expansion (0.49)	woman (0.43)	escape (0.35)	nation (0.34)
comp.graphics	interrupt (0.83)	driver (0.80)	version (0.75)	video (0.55)	network (0.54)
comp.windows.x	domain (0.48)	information (0.39)	source (0.34)	core (0.31)	message (0.30)

TABLE 7: Interpretation of SR-RNN ($k = 20$ centroids) model trained on the 20NewsGroup.

misc.forsale	sci.space
for sale one complete set UNK equipment including base unit portable transmitter UNK plus days free UNK service description item convenient secure anyone whose home broken whose parents live alone children elderly parents UNK heart attack stroke temporarily permanently disabled superior features allows talk UNK center using transmitter help sent soon possible allows personal freedom independence deal item worth us open market asking best offer interested please email UNK UNK stanford edu call will send UNK delivery relevant documents	(...) long term planetary monitoring mission occasional chance UNK something like top UNK mission like galileo UNK it unlikely much happening pluto would worth monitoring UNK difficult mission fly without new propulsion technology something planetary community firmly UNK UNK UNK the combined need arrive pluto within reasonable amount time kill nearly cruise velocity settle orbit beyond reasonably done current UNK propulsion most done well earth the things done better voyager like spacecraft UNK need enter orbit around planet

TABLE 8: Explaining the SR-RNN prediction on two 20NewsGroup test samples (stronger highlight indicates higher transition probability).

Prediction	Test samples
Negative	no comment - stupid movie , acting average or worse ... screenplay - no sense at all ... skip it !
Positive	i thought this was one of those really great films to see with a bunch of close friends . i laughed and cried and laughed and cried at the same time

TABLE 9: For a negative (top) and positive (bottom) IMDB prediction, the SR-RNN highlights the words according to the probability of transitioning to the negative(top)/positive(bottom) centroid (stronger highlight indicates higher transition probability).

20NewsGroup which are categorized to "misc.forsale" and "sci.space" categories. The highlighted words "sale, asking, offer delivery" and "planetary, orbit, spacecraft, planet" are highly associative to the RNN predictions.

Table 9 presents the predictions of two examples (a positive sample and a negative sample). At each time step, the input words are transitioned between the learned "positive" and "negative" centroids. In the negative example, the representative negative words "no, stupid, worse, skip" are given high transition probabilities to the negative centroid. Similarly, in the bottom example, the words "great" and "laughed" are associated with the positive centroid.

7 Discussion

We believe that the probabilistic finite state transition has some additional nice properties, which we discuss in the following.

N-gram Phase Extraction. With probabilistic finite state transition, we can extend word-level to phrase-level interpretability and explainability. This can be achieved by maintaining an attention window with a size of n on the input sequence. The phrases with the highest mean transition probability can be extracted. Table 10 demonstrates the extracted n-gram phrases for explaining predictions on the IMDB dataset.

2-gram	4-gram
worst movie	terrible , terrible ,
bad choice	not a decent performance
odd details	writing : 1 / 10
incredibly awful	movie is extremely boring
wasted moments	overacting , see it
great acting	10 / 10 .
superbly crafted	very impressed with this
10 !	movie . great storyline
absolutely incredible	extremely well composed movie
exceptional .	great cast all around

TABLE 10: The n-gram phrases extracted from 1K random IMDB samples for positive (green) and negative (red) predictions.

Transitions as Features. The pattern of probabilistic finite state transition that the SR-RNN learnt can be used as a representation. Figure 9 shows the t-SNE [40] of finite state transition probabilities for test samples. We find that the transition probabilities of categories are discriminative.

Dimensionality Reduction. Note that, Figure 9 is not the visualization of d-dimension intermediate representation $X \in \mathbb{R}^d$, but the state transition probabilities $P \in \mathbb{R}^k$ of pixel sequences over k centroids. In most cases $k \ll d$ (in this case, $k = 10, d = 256$), which also suggests a possibility of using probabilistic finite state transition as dimensionality reduction method.

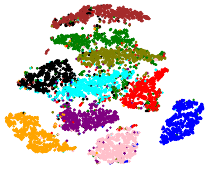


Fig. 9: t-SNE visualization of the extracted finite state transition probabilities for test samples with SR-RNNs ($k = 10$) trained on the MNIST.

8 Conclusion

State-regularization provides new mechanisms for understanding the workings of RNNs. Inspired by recent DFA extraction work [62], our work simplifies the extraction approach by directly learning a finite set of states and an interpretable state transition dynamic. Even on realistic tasks, such as sentiment analysis, exploring the learned centroids and the transition behavior of SR-RNNs makes for more interpretable RNN models whithout sacrificing accuracy: a single-layer SR-RNNs is competitive with state-of-the-art methods. The purpose of our work is not to surpass all existing state of the art methods but to gain a deeper understanding of the dynamics of RNNs.

State-regularized RNNs operate more like automata with external memory and less like DFAs. This results in a markedly improved extrapolation behavior on several datasets. We do not claim, however, that SR-RNNs are a panacea for all problems associated with RNNs did. For instance, we could not observe an improved convergence of SR-RNNs. Sometimes SR-RNNs converged faster, sometimes vanilla RNNs. While we have mentioned that the computational overhead of SR-RNNs is modest, it still exists, and this might exacerbate the problem that RNNs often take a long to be trained and tuned. We plan to investigate variants of state regularization and the ways in which it could improve differentiable computers with RNN controllers in the future.

References

[1] Martin Arjovsky, Amar Shah, and Yoshua Bengio. Unitary evolution recurrent neural networks. In *ICML*, pages 1120–1128, 2016.

[2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *ICLR*, 2015.

[3] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *CoRR*, abs/1803.01271, 2018.

[4] Justin Bayer and Christian Osendorfer. Learning stochastic recurrent networks. *arXiv preprint arXiv:1411.7610*, 2014.

[5] David M Blei, Andrew Y Ng, and Michael I Jordan. Latent dirichlet allocation. *JMLR*, 2003.

[6] Víctor Campos, Brendan Jou, Xavier Giró-i Nieto, Jordi Torres, and Shih-Fu Chang. Skip rnn: Learning to skip state updates in recurrent neural networks. *ICLR*, 2018.

[7] Shiyu Chang, Yang Zhang, Wei Han, Mo Yu, Xiaoxiao Guo, Wei Tan, Xiaodong Cui, Michael Witbrock, Mark A Hasegawa-Johnson, and Thomas S Huang. Dilated recurrent neural networks. In *NIPS*, pages 77–87, 2017.

[8] Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555*, 2014.

[9] Tim Cooijmans, Nicolas Ballas, César Laurent, Çağlar Gülçehre, and Aaron Courville. Recurrent batch normalization. *ICLR*, 2017.

[10] George E Dahl, Ryan P Adams, and Hugo Larochelle. Training restricted boltzmann machines on word observations. *ICML*, 2012.

[11] Andrew M Dai and Quoc V Le. Semi-supervised sequence learning. In *NIPS*, pages 3079–3087, 2015.

[12] Ivo Danihelka, Greg Wayne, Benigno Uria, Nal Kalchbrenner, and Alex Graves. Associative long short-term memory. In *ICML*, pages 1986–1994, 2016.

[13] Michal Daniluk, Tim Rocktäschel, Johannes Welbl, and Sebastian Riedel. Frustratingly short attention spans in neural language modeling. 2017.

[14] Adji B Dieng, Rajesh Ranganath, Jaan Altosaar, and David M Blei. Noisin: Unbiased regularization for recurrent neural networks. *ICML*, 2018.

[15] Jeffrey L Elman. Finding structure in time. *Cognitive science*, 14(2):179–211, 1990.

[16] Jakob N Foerster, Justin Gilmer, Jascha Sohl-Dickstein, Jan Chorowski, and David Sussillo. Input switched affine networks: An rnn architecture designed for interpretability. In *ICML*, pages 1136–1145. JMLR. org, 2017.

[17] Marco Fraccaro, Søren Kaae Sø nderby, Ulrich Paquet, and Ole Winther. Sequential neural models with stochastic layers. In *NeurIPS*, pages 2199–2207. 2016.

[18] Paolo Frasconi and Yoshua Bengio. An em approach to grammatical inference: input/output hmms. In *ICPR*, pages 289–294. IEEE, 1994.

[19] Yarin Gal and Zoubin Ghahramani. A theoretically grounded application of dropout in recurrent neural networks. In *NeurIPS*, pages 1019–1027, 2016.

[20] Felix A Gers and E Schmidhuber. Lstm recurrent networks learn simple context-free and context-sensitive languages. *IEEE Transactions on Neural Networks*, 12(6):1333–1340, 2001.

[21] Felix A. Gers and Jürgen Schmidhuber. Recurrent nets that time and count. In *IJCNN (3)*, pages 189–194, 2000.

[22] C Lee Giles, D Chen, CB Miller, HH Chen, GZ Sun, and YC Lee. Second-order recurrent neural networks for grammatical inference. In *IJCNN*, volume 2, pages 273–281. IEEE, 1991.

[23] Anirudh Goyal, Alessandro Sordoni, Marc-Alexandre Côté, Nan Ke, and Yoshua Bengio. Z-forcing: Training stochastic recurrent networks. In *NeurIPS*, pages 6713–6723. 2017.

[24] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.

[25] Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwinska, Sergio Gomez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adrià Puigdomènech Badia, Karl Moritz Hermann, Yori Zwols, Georg Ostrovski, Adam Cain, Helen King, Christopher Summerfield, Phil Blunsom, Koray Kavukcuoglu, and Demis Hassabis. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476, 2016.

[26] Edward Grefenstette, Karl Moritz Hermann, Mustafa Suleyman, and Phil Blunsom. Learning to transduce with unbounded memory. In *NeurIPS*, pages 1828–1836, 2015.

[27] Emil Julius Gumbel. Statistical Theory of Extreme Values and Some Practical Applications. A Series of Lectures. *Number 33. US Govt. Print. Office*, 1954.

[28] Yiding Hao, William Merrill, Dana Angluin, Robert Frank, Noah Amsel, Andrew Benz, and Simon Mendelsohn. Context-free transductions with neural stacks. In *EMNLP workshops*, 2018.

[29] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, 1997.

[30] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. In *ICLR, year = 2017, url = https://openreview.net/forum?id=rkE3y85ee,*.

[31] Rie Johnson and Tong Zhang. Effective use of word order for text categorization with convolutional neural networks. *NAACL HLT*, 2015.

[32] Andrej Karpathy, Justin Johnson, and Li Fei-Fei. Visualizing and understanding recurrent networks. *ICLR workshop*, 2016.

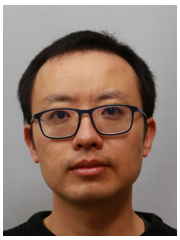
[33] Alex Kendall and Yarin Gal. What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision? In *NIPS*, 2017.

[34] Teuvo Kohonen. Learning vector quantization. In *Self-organizing maps*, pages 175–189. Springer, 1995.

[35] David Krueger, Tegan Maharaj, János Kramár, Mohammad Pezeshki, Nicolas Ballas, Nan Rosemary Ke, Anirudh Goyal, Yoshua Bengio, Aaron Courville, and Chris Pal. Zoneout: Regularizing rnns by randomly preserving hidden activations. *ICLR*, 2017.

[36] Quoc V Le, Navdeep Jaitly, and Geoffrey E Hinton. A simple way to initialize recurrent networks of rectified linear units. *arXiv preprint arXiv:1504.00941*, 2015.

[37] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
[38] Jiwei Li, Xinlei Chen, Eduard Hovy, and Dan Jurafsky. Visualizing and understanding neural models in nlp. In *NAACL-HLT*, pages 681–691, 2016.
[39] Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. Learning word vectors for sentiment analysis. In *ACL-HLT*, pages 142–150, Portland, Oregon, USA, June 2011. Association for Computational Linguistics.
[40] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *JMLR*, 9(Nov):2579–2605, 2008.
[41] Stephen Merity, Nitish Shirish Keskar, and Richard Socher. Regularizing and optimizing lstm language models. *ICLR*, 2018.
[42] John Miller and Moritz Hardt. When recurrent models don't need to be recurrent. *CoRR*, abs/1805.10369, 2018.
[43] Grégoire Montavon, Wojciech Samek, and Klaus-Robert Müller. Methods for interpreting and understanding deep neural networks. *Digital Signal Processing*, 73:1–15, 2018.
[44] W James Murdoch and Arthur Szlam. Automatic rule extraction from long short term memory networks. In *ICLR*, 2017.
[45] Jiahuan Pei, Cheng Wang, and György Szarvas. Transformer uncertainty estimation with hierarchical stochastic attention. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 11147–11155, 2022.
[46] Ingo Schellhammer, Joachim Diederich, Michael Towsey, and Claudia Brugman. Knowledge extraction and recurrent neural networks: An analysis of an elman network trained on a natural language learning task. In *Proceedings of the Joint Conferences on New Methods in Language Processing and Computational Natural Language Learning*, pages 73–78, 1998.
[47] Jürgen Schmidhuber, F Gers, and Douglas Eck. Learning nonregular languages: A comparison of simple recurrent networks and lstm. *Neural Computation*, 14(9):2039–2041, 2002.
[48] Hava T Siegelmann. *Neural networks and analog computation: beyond the Turing limit*. Springer Science & Business Media, 2012.
[49] Hava T Siegelmann and Eduardo D Sontag. On the computational power of neural nets. In *Proceedings of the fifth annual workshop on Computational learning theory*, pages 440–449. ACM, 1992.
[50] Hava T Siegelmann and Eduardo D Sontag. Analog computation via neural networks. *Theoretical Computer Science*, 131(2):331–360, 1994.
[51] Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. *arXiv preprint arXiv:1312.6034*, 2013.
[52] Hendrik Strobelt, Sebastian Gehrmann, Bernd Huber, Hanspeter Pfister, Alexander M Rush, et al. Visual analysis of hidden state dynamics in recurrent neural networks. *CoRR, abs/1606.07461*, 2016.
[53] Theano Development Team. Theano: A Python framework for fast computation of mathematical expressions. *arXiv e-prints*, abs/1605.02688, 2016.
[54] Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude. *COURSERA: Neural networks for machine learning*, 4(2):26–31, 2012.
[55] M. Tomita. Dynamic construction of finite automata from examples using hill-climbing. In *Proceedings of the Fourth Annual Conference of the Cognitive Science Society*, pages 105–108, 1982.
[56] Trieu H Trinh, Andrew M Dai, Thang Luong, and Quoc V Le. Learning longer-term dependencies in rnns with auxiliary losses. *ICML*, 2018.
[57] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *NIPS*, pages 5998–6008. 2017.
[58] Cheng Wang, Carolin Lawrence, and Mathias Niepert. Uncertainty estimation and calibration with finite-state probabilistic {rnn}s. In *ICLR*, 2021.
[59] Cheng Wang and Mathias Niepert. State-regularized recurrent neural networks. In *ICML*, pages 6596–6606. PMLR, 2019.
[60] Qinglong Wang, Kaixuan Zhang, II Ororbia, G Alexander, Xinyu Xing, Xue Liu, and C Lee Giles. A comparison of rule extraction for different recurrent neural network models and grammatical complexity. *arXiv preprint arXiv:1801.05420*, 2018.
[61] Qinglong Wang, Kaixuan Zhang, Alexander G. Ororbia II, Xinyu Xing, Xue Liu, and C. Lee Giles. An empirical evaluation of rule extraction from recurrent neural networks. *Neural Computation*, 30(9):2568–2591, 2018.
[62] Gail Weiss, Yoav Goldberg, and Eran Yahav. Extracting automata from recurrent neural networks using queries and counterexamples. In *ICML*, volume 80, pages 5247–5256, 2018.
[63] Gail Weiss, Yoav Goldberg, and Eran Yahav. On the practical computational power of finite precision rnns for language recognition. 2018.
[64] Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. 2015.
[65] Scott Wisdom, Thomas Powers, John Hershey, Jonathan Le Roux, and Les Atlas. Full-capacity unitary recurrent neural networks. In D. D. Lee, M. Sugiyama, U. V. Luxburg, I. Guyon, and R. Garnett, editors, *NIPS*, pages 4880–4888. 2016.
[66] Adams Wei Yu, Hongrae Lee, and Quoc V Le. Learning to skim text. *ACL*, 2017.
[67] Wojciech Zaremba and Ilya Sutskever. Learning to execute. *CoRR*, abs/1410.4615, 2014.
[68] Wojciech Zaremba, Ilya Sutskever, and Oriol Vinyals. Recurrent neural network regularization. *arXiv preprint arXiv:1409.2329*, 2014.
[69] Matthew D Zeiler. Adadelta: an adaptive learning rate method. *arXiv preprint arXiv:1212.5701*, 2012.
[70] Matthew D Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. In *ECCV*, pages 818–833. Springer, 2014.
[71] Zheng Zeng, Rodney M Goodman, and Padhraic Smyth. Learning finite state machines with self-clustering recurrent networks. *Neural Computation*, 5(6):976–990, 1993.
[72] Quanshi Zhang, Ying Nian Wu, and Song-Chun Zhu. Interpretable convolutional neural networks. In *CVPR*, pages 8827–8836, 2018.
[73] Saizheng Zhang, Yuhuai Wu, Tong Che, Zhouhan Lin, Roland Memisevic, Ruslan R Salakhutdinov, and Yoshua Bengio. Architectural complexity measures of recurrent neural networks. In *NIPS*, pages 1822–1830, 2016.
[74] Julian Georg Zilly, Rupesh Kumar Srivastava, Jan Koutník, and Jürgen Schmidhuber. Recurrent highway networks. In *ICML*, pages 4189–4198. JMLR. org, 2017.



Cheng Wang is a machine learning scientist. He received his Dr. rer. nat. degree from Hasso Plattner Institute, the University of Potsdam (2017). His research interests are machine learning, multimodal deep learning and recurrent neural networks with applications in language and vision, information retrieval tasks. He is a PC member of ICML, NeurPS, ICLR, NAACL, ACL, EMNLP, AAAI, IJCAI, ACMMM and an invited reviewer of AIJ, IEEE TNNLS, IEEE TIP, IEEE TKDE, IEEE TMM etc.. He is IEEE and ACM member.



Carolin Lawrence is a manager at NEC Laboratories Europe. She received her PhD with the highest distinction in Computational Linguistics from Heidelberg University, Germany (2019). Her research focus includes explainable AI, human-centric AI, NLP and knowledge graphs. She is a senior programm committee member of top-tier NLP conferences (ACL, NAACL, EMNLP). She won the outstanding paper award at the leading conference for knowledge graphs, Automated Knowledge Base Construction (AKBC), in 2021.



Mathias Niepert is a professor at the University of Stuttgart, a faculty member of the Max Planck Research School for Intelligent Systems and ELLIS, and Chief Scientific Advisor at NEC Labs Europe. He received his PhD from Indiana University (2009) and was a postdoctoral researcher at the University of Washington, Seattle. His research interests include deep geometric learning, unsupervised and semi-supervised learning, and probabilistic graphical models. He has won several best paper awards and grants such as a Google Research Award. He is a PC member and/or area chair of top conferences such as ICML, NeurIPS, UAI, ICLR, AAAI and IJCAI.

SUPPLEMENTARY MATERIALS

Proofs of Theorems 4.1 and 4.2

Theorem 4.1: **The state transition behavior of an SR-RNN without ∞-memory using equation 16 is identical to that of a probabilistic finite automaton.**

Proof. The state transition function δ of a probabilistic finite state machine is identical to that of a finite deterministic automaton (see section 2) with the exception that it returns a probability distribution over states. For every state q and every input token a the transition mapping δ returns a probability distribution

$$\boldsymbol{\alpha} = (\alpha_1, ..., \alpha_k) = \delta(q, a) \tag{29}$$

that assigns a fixed probability to each possible state $q \in \mathcal{Q}$ with $|\mathcal{Q}| = k$. The automaton transitions to the next state according to this distribution. Since by assumption the SR-RNN is using equation 16, we only have to show that the probability distribution over states computed by the stochastic component of an SR-RNN without ∞-memory is identical for every state q and every input token a irrespective of the previous input sequence $\mathbf{a} = \{a_1, ..., a_n\}$ and corresponding state transition history $\mathbf{q} = \{q_1, ..., q_n\}$.

$$\delta(q_1, ..., q_n, q, a_1, ..., a_n, a) = \delta(q, a) \tag{30}$$

More formally, for every pair of input token sequences $\mathbf{a}_1$ and $\mathbf{a}_2$ with corresponding pair of resulting state sequences $\mathbf{q}_1 = (q_{i_1}, ..., q_{i_n}, q)$ and $\mathbf{q}_2 = (q_{j_1}, ..., q_{j_m}, q)$ in SR-RNN without ∞-memory, we have to prove, for every token $a \in \Sigma$, that $\boldsymbol{\alpha}_1$ and $\boldsymbol{\alpha}_2$, the probability distributions over the states returned by the stochastic component for state q and input token a, are identical.

$$\delta(q_{i_1}, ..., q_{i_n}, q, a) = \delta(q_{j_1}, ..., q_{j_m}, q, a) \tag{31}$$

Now, since the RNN is, by assumption, without ∞-memory, we have for both $\mathbf{a}_1, \mathbf{q}_1$ and $\mathbf{a}_2, \mathbf{q}_2$ that the only inputs to the RNN cell are exactly the centroid $\mathbf{s}_q$ corresponding to state q and the vector representation of token a. Hence, under the assumption that the parameter weights of the RNN are the same for both state sequences $\mathbf{q}_1$ and $\mathbf{q}_2$, we have that the output $\mathbf{u}$ of the recurrent component (the base RNN cell) is identical for $\mathbf{q}_1$ and $\mathbf{q}_2$. Finally, since by assumption the centroids $\mathbf{s}_1, ..., \mathbf{s}_k$ are fixed, we have that the returned probability distributions $\boldsymbol{\alpha}_1$ and $\boldsymbol{\alpha}_2$ are identical. Hence, the transition behavior of SR-RNN without ∞-memory is identical to that of a probabilistic finite automaton. □

Theorem 4.2: **For $\tau \to 0$ the state transition behavior of an SR-RNN without ∞-memory (using equations 16 or 20) is equivalent to that of a deterministic finite automaton.**

Proof. Let us consider the softmax function with temperature parameter τ

$$\alpha_i = \frac{\exp(b_i/\tau)}{\sum_{i=1}^{k} \exp(b_i/\tau)}$$

for $1 \leq i \leq k$. SR-RNNs use this softmax function to normalize the scores (from a dot product) into a probability distribution. First, we show that for $\tau \to 0^+$, that there is exactly one $M \in \{1, ..., k\}$ such that $\alpha_M = 1$ and $\alpha_i = 0$ for all $i \in \{1, ..., k\}$ with $i \neq M$. Without loss of generality, we assume that there is a $M \in \{1, ..., k\}$ such that $b_M > b_i$ for all $i \in \{1, ..., k\}, i \neq M$. Hence, we can write for $\epsilon_1, ..., \epsilon_k > 0$ as shown in equation(26,27,28).

Now, for $\tau \to 0$ we have that $\alpha_M \to 1$ and for all other $i \neq M$ we have that $\alpha_i \to 0$. Hence, the probability distribution $\boldsymbol{\alpha}$ of the SR-RNN is always the one-hot encoding of a particular centroid.

By an argument analog to the one we have made for Theorem 4.1, we can prove that for every state $q \in \mathcal{Q}$ and every input token $a \in \Sigma$, the probability distribution $\boldsymbol{\alpha}$ of the SR-RNN is the same irrespective of the previous input sequences and visited states. Finally, by plugging in the one-hot encoding $\boldsymbol{\alpha}$ in both equations 16 and 20, we can conclude that the transition function of an SR-RNN without ∞-memory is identical to that of a DFA, because we always chose exactly one new state. □

$$\alpha_i = \frac{\exp(b_i/\tau)}{\exp((b_M - \epsilon_1)/\tau) + ... + \exp(b_M/\tau) + ... + \exp((b_M - \epsilon_k)/\tau)} \tag{26}$$

$$= \frac{\exp(b_i/\tau)}{\exp(b_M/\tau)\exp(\epsilon_1/\tau)^{-1} + ... + \exp(b_M/\tau) + ... + \exp(b_M/\tau)\exp(\epsilon_k/\tau)^{-1}} \tag{27}$$

$$= \frac{\exp(b_i/\tau)}{\exp(b_M/\tau)\left[\exp(\epsilon_1/\tau)^{-1} + ... + 1 + ... + \exp(\epsilon_k/\tau)^{-1}\right]}. \tag{28}$$